

Competitive Programming

Section 9 Editorial: Greedy I - Local Choices And Exchange

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	4
2	A. Activity Selection	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	6
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	7
2.8	C++ implementation notes	7
2.9	Python implementation notes	7
2.10	Full C++ solution	7
2.11	Full Python solution	8
2.12	Common wrong answers	9
2.13	Tests that matter	9
3	B. Rescue Boats	10
3.1	Restatement	10
3.2	Constraints and traps	10
3.3	Observations	10
3.4	Sample walk-through	10
3.5	Algorithm	11
3.6	Correctness proof	11
3.7	Complexity	12
3.8	C++ implementation notes	12
3.9	Python implementation notes	12
3.10	Full C++ solution	12
3.11	Full Python solution	13
3.12	Common wrong answers	14
3.13	Tests that matter	14
4	C. Deadline Schedule	15
4.1	Restatement	15
4.2	Constraints and traps	15
4.3	Observations	15
4.4	Heap reminder	16
4.5	Sample walk-through	16
4.6	Algorithm	17
4.7	Correctness proof	17
4.8	Complexity	18
4.9	C++ implementation notes	18
4.10	Python implementation notes	18
4.11	Full C++ solution	18

4.12 Full Python solution	19
4.13 Common wrong answers	20
4.14 Tests that matter	20
5 Stress testing greedy	21
6 What to carry forward	22

1 Editorial overview

This section has three local greedy problems.

The important skill is not memorizing the final code. The important skill is learning to defend the local choice.

Each solution below follows the same pattern:

1. Restate the optimization problem.
2. Identify the local conflict.
3. Choose a local action.
4. Prove the action by exchange or invariant.
5. Implement the scan.

2 A. Activity Selection

2.1 Restatement

Given intervals $[s_i, e_i)$, choose the largest possible number of intervals so that no chosen intervals overlap. Two intervals are compatible when the earlier one ends at or before the later one starts.

Only the maximum count is required.

2.2 Constraints and traps

The total number of intervals is large, so subset enumeration is impossible.

Common traps:

- treating touching intervals as overlapping;
- sorting by start time instead of end time;
- using the shortest interval first;
- forgetting negative starts or large coordinates.

Because intervals are half-open, these two intervals are compatible:

```
[1, 3) and [3, 5)
```

The condition in code is:

```
start >= last_end
```

2.3 Observations

Suppose we choose an interval first. After that, every interval that starts before its end becomes impossible.

To maximize future options, we want the first chosen interval to end as early as possible.

That suggests:

```
Sort intervals by ending time.  
Scan in that order.  
Take an interval if it starts after the last chosen interval ends.
```

2.4 Sample walk-through

Input intervals:

```
[1,3), [2,5), [4,7), [6,9)
```

Sorted by end time:

```
[1,3), [2,5), [4,7), [6,9)
```

Scan:

```
last_end = -infinity
take [1,3), last_end = 3
skip [2,5), because 2 < 3
take [4,7), last_end = 7
skip [6,9), because 6 < 7
answer = 2
```

2.5 Algorithm

```
answer = 0
last_end = -infinity
sort intervals by (end, start)

for each interval (start, end):
    if start >= last_end:
        answer += 1
        last_end = end

print answer
```

2.6 Correctness proof

We prove that the algorithm returns the maximum possible number of intervals.

Lemma 1. Among all intervals available at any step, choosing the interval with the earliest ending time is safe.

Proof. Let g be the available interval with the earliest ending time. Consider an optimal solution for the remaining problem. Let x be the first interval in that optimal solution.

Since g has the earliest ending time, we have:

```
end(g) <= end(x)
```

Replace x by g . No later interval in the optimal solution starts before $\text{end}(x)$, so no later interval starts before $\text{end}(g)$ either. Therefore the replacement keeps the solution valid. The number of intervals is unchanged.

So there exists an optimal solution that chooses g first. This proves the greedy choice is safe. $\square$

Lemma 2. After the algorithm chooses an interval, the remaining problem is again an activity selection problem on the intervals that start after the chosen interval ends.

Proof. Any interval that starts before last_end conflicts with the chosen interval and cannot be part of any solution extending the current choice. Every other interval is compatible with the chosen interval, and future compatibility depends only on intervals chosen after it. $\square$

Theorem. The algorithm returns an optimal answer.

Proof. By Lemma 1, the first greedy interval can be the first interval of some optimal solution. By Lemma 2, after taking it, the remaining subproblem has the same form. Repeating this argument for each greedy choice shows that there is an optimal solution containing every interval chosen by the algorithm. When the scan ends, no compatible intervals remain, so the algorithm's count is optimal. $\square$

2.7 Complexity

Sorting dominates.

```
Time:  $O(n \log n)$   
Memory:  $O(n)$ 
```

2.8 C++ implementation notes

Use a vector of pairs $(\text{start}, \text{end})$, but sort by end .

Use a very small last_end sentinel:

```
long long last_end = numeric_limits<long long>::min() / 4;
```

2.9 Python implementation notes

Sort with:

```
intervals.sort(key=lambda p: (p[1], p[0]))
```

The starting sentinel can be -10^{**30} .

2.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int solve_case(vector<pair<long long, long long>> intervals) {
    sort(intervals.begin(), intervals.end(), [](const auto &a, const auto &b) {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first;
    });

    int count = 0;
    long long last_end = numeric_limits<long long>::min() / 4;
    for (auto [start, end] : intervals) {
        if (start >= last_end) {
            ++count;
            last_end = end;
        }
    }
    return count;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[start, end] : intervals) cin >> start >> end;
        cout << solve_case(intervals) << '\n';
    }

    return 0;
}

```

2.11 Full Python solution

```

import sys

def solve_case(intervals: list[tuple[int, int]]) -> int:
    intervals.sort(key=lambda p: (p[1], p[0]))
    count = 0
    last_end = -10**30
    for start, end in intervals:
        if start >= last_end:

```

```

        count += 1
        last_end = end
    return count

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        intervals: list[tuple[int, int]] = []
        for _ in range(n):
            start = data[idx]
            end = data[idx + 1]
            idx += 2
            intervals.append((start, end))
        out.append(str(solve_case(intervals)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

2.12 Common wrong answers

- Sorting by start time.
- Using `start > last_end` instead of `start >= last_end`.
- Choosing shortest interval first.
- Updating `last_end` when skipping an interval.

2.13 Tests that matter

Touching intervals:
`[1,2), [2,3), [3,4) -> 3`

One interval containing all others:
`[1,10), [2,3), [3,4), [4,5) -> 3`

All intervals overlap:
`[1,5), [2,6), [3,7) -> 1`

3 B. Rescue Boats

3.1 Restatement

There are n people. A boat carries at most two people and total weight at most L . Every person must be transported. Minimize the number of boats.

3.2 Constraints and traps

The large input size rules out matching-style brute force.

Common traps:

- pairing adjacent light people first;
- trying to fill each boat as much as possible from the light side;
- forgetting that a boat holds at most two people;
- mishandling the final unpaired person.

3.3 Observations

Sort the weights.

Consider the heaviest remaining person. This person must either:

1. ride alone; or
2. ride with exactly one other remaining person.

If the lightest remaining person cannot ride with the heaviest, then no one can ride with the heaviest. The heaviest must ride alone.

If the lightest can ride with the heaviest, pairing them is safe.

This gives a two-pointer algorithm.

3.4 Sample walk-through

Weights:

```
3 2 2 1
L = 5
```

Sorted:

```
1 2 2 3
```

Trace:

```
left = 0, right = 3
1 + 3 <= 5, pair them. boats = 1

left = 1, right = 2
2 + 2 <= 5, pair them. boats = 2
```

Answer: 2.

3.5 Algorithm

```
sort weights
left = 0
right = n - 1
boats = 0

while left <= right:
    if weights[left] + weights[right] <= L:
        left += 1
        right -= 1
        boats += 1

print boats
```

Every loop iteration assigns the heaviest remaining person to a boat.

3.6 Correctness proof

Lemma 1. If the lightest remaining person cannot ride with the heaviest remaining person, then the heaviest remaining person must ride alone in every valid solution.

Proof. Every other remaining person is at least as heavy as the lightest. If even the lightest plus the heaviest exceeds L , then any other partner plus the heaviest also exceeds L . Since a boat can carry at most two people, the heaviest must ride alone. $\square$

Lemma 2. If the lightest remaining person can ride with the heaviest remaining person, then there exists an optimal solution in which they ride together.

Proof. Consider an optimal solution.

If the lightest and heaviest already ride together, we are done.

Otherwise, the heaviest either rides alone or with some person x . Move the lightest person into the heaviest person's boat. This is feasible by assumption.

If the heaviest originally rode with x , then x is no lighter than the lightest person. Put x where the lightest person was. If the lightest was alone, then x can be alone. If the lightest had a partner y , then:

`weight(x)` may be larger than `weight(lightest)`

So that direct swap might fail. Use a simpler exchange: pair the lightest with the heaviest and leave the rest of the people assigned using no more boats than before. The heaviest's old boat disappears or loses its other passenger, and the lightest's old boat can only become easier after removing the lightest. No extra boat is needed. Therefore an optimal solution exists with the greedy pair. $\square$

Lemma 3. Each loop iteration makes a choice that appears in some optimal solution for the current remaining people.

Proof. If the lightest cannot pair with the heaviest, Lemma 1 applies. If the lightest can pair with the heaviest, Lemma 2 applies. $\square$

Theorem. The algorithm uses the minimum possible number of boats.

Proof. Each iteration assigns the heaviest remaining person in a way that some optimal solution can also assign that person. Removing the assigned person or pair leaves the same problem on fewer people. By induction over the number of remaining people, the algorithm is optimal. $\square$

3.7 Complexity

Time: $O(n \log n)$
Memory: $O(n)$

The two-pointer scan itself is linear.

3.8 C++ implementation notes

Use integer weights if constraints allow, but `long long` is also fine. The condition should be checked before moving `right`.

3.9 Python implementation notes

The loop condition is `while left <= right`, not `left < right`, because one remaining person still needs one boat.

3.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int solve_case(vector<int> weights, int limit) {
    sort(weights.begin(), weights.end());
    int left = 0;
    int right = (int)weights.size() - 1;
    int boats = 0;
    while (left <= right) {
        if (weights[left] + weights[right] <= limit) {
            ++left;
        }
        --right;
        ++boats;
    }
    return boats;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, limit;
        cin >> n >> limit;
        vector<int> weights(n);
        for (int &w : weights) cin >> w;
        cout << solve_case(weights, limit) << '\n';
    }

    return 0;
}

```

3.11 Full Python solution

```

import sys

def solve_case(weights: list[int], limit: int) -> int:
    weights.sort()
    left = 0
    right = len(weights) - 1
    boats = 0
    while left <= right:
        if weights[left] + weights[right] <= limit:
            left += 1

```

```

        right -= 1
        boats += 1
    return boats

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        limit = data[idx + 1]
        idx += 2
        weights = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(weights, limit)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

3.12 Common wrong answers

- Pairing lightest with second lightest.
- Using a multiset but choosing arbitrary partners.
- Forgetting that every heaviest person consumes exactly one boat.
- Returning $\text{ceil}(n / 2)$ when weights matter.

3.13 Tests that matter

All can pair:

1 2 2 3, L = 5 -> 2

Nobody can pair with the heaviest:

3 3 3 3, L = 3 -> 4

Pairing adjacent is bad:

1 2 8 9, L = 10 -> 2

4 C. Deadline Schedule

4.1 Restatement

There are optional tasks. Task i takes p_i time and has deadline d_i . Choose the largest number of tasks that can be completed on time when processed one after another from time 0.

You may choose the subset. You do not need to output the order.

4.2 Constraints and traps

The total number of tasks is large. We need an $O(n \log n)$ solution.

Common traps:

- sorting only by duration;
- accepting a task forever once it is chosen;
- checking only the final total time instead of every deadline;
- using a min-heap when a max-heap is needed.

4.3 Observations

If a set of tasks can be completed by their deadlines, then ordering those tasks by nondecreasing deadline is safe. This is the standard earliest-deadline order.

So we process tasks sorted by deadline.

When we process a task with deadline d , all chosen tasks so far have deadline at most d . The only question is:

```
Is the total chosen duration <= d?
```

If yes, keep the new task.

If no, we have chosen too much work among tasks whose deadlines are already active. To keep the number of chosen tasks as large as possible, remove exactly one chosen task. Removing the longest duration gives the smallest remaining total time.

4.4 Heap reminder

The algorithm needs to repeatedly remove the longest duration among currently chosen tasks.

A max-heap supports:

```
push(duration)      O(log n)
top largest value    O(1)
pop largest value    O(log n)
```

In C++, `priority_queue<long long>` is already a max-heap:

```
priority_queue<long long> chosen;
chosen.push(duration);
total -= chosen.top();
chosen.pop();
```

Python's `heapq` is a min-heap, so the solution stores negative durations:

```
heapq.heappush(chosen, -duration)
total += heapq.heappop(chosen)
```

The popped value is negative, so adding it decreases `total` by the longest duration.

4.5 Sample walk-through

Tasks:

duration	deadline
3	5
2	6
4	6
1	3

Sorted by deadline:

```
(1,3), (3,5), (2,6), (4,6)
```

Trace:

```
take 1: total = 1 <= 3, heap = [1]
take 3: total = 4 <= 5, heap = [3, 1]
take 2: total = 6 <= 6, heap = [3, 1, 2]
take 4: total = 10 > 6
drop longest 4: total = 6, heap = [3, 1, 2]
```

Answer: 3.

4.6 Algorithm

```
sort tasks by (deadline, duration)
total = 0
max_heap = empty

for (duration, deadline) in tasks:
    total += duration
    push duration into max_heap
    if total > deadline:
        total -= max_heap.pop_max()

print size(max_heap)
```

4.7 Correctness proof

We use an invariant.

After processing the first i tasks in deadline order, the heap contains a maximum-size subset of those i tasks that can be completed by their deadlines. Among all subsets of that size, it has minimum total duration.

Base case. Before any task is processed, the empty set is maximum-size and has total duration 0.

Inductive step. Suppose the invariant holds before processing task i . Add task i to the heap.

If the new total duration is at most task i 's deadline, the chosen set is feasible. Its size is one larger than before, so no other subset of processed tasks can be larger: any larger subset would include more than one new task, which is impossible because only one task was added. The invariant holds.

If the new total duration exceeds task i 's deadline, then the current chosen set has one task too many or contains too much total duration. We remove the longest duration among chosen tasks.

Why is removing the longest safe?

Consider any subset with the same number of tasks after one removal from the tentative chosen set. The subset obtained by removing the longest task has the minimum possible total duration among them. Therefore, if any subset of this size can satisfy the current deadline, this one is at least as easy to schedule.

The size returns to the previous heap size. By the induction hypothesis, before adding task i there was no feasible subset larger than that size among earlier tasks. Since the tentative larger set violated the current deadline and removing one task gives the best possible total duration among such removals, keeping this size is optimal after processing task i .

The heap therefore again stores a maximum-size feasible subset, with minimum total duration among maximum-size subsets. $\square$

Theorem. After all tasks are processed, the heap size is the maximum number of tasks that can be completed on time.

Proof. The invariant holds after every processed task. After the last task, all tasks have been processed, so the heap is a maximum-size feasible subset of all tasks. $\square$

4.8 Complexity

Sorting costs $O(n \log n)$. Each task is pushed once and popped at most once.

```
Time:  $O(n \log n)$ 
Memory:  $O(n)$ 
```

4.9 C++ implementation notes

Use `priority_queue<long long>` for a max-heap.

When total exceeds the current deadline:

```
total -= chosen.top();
chosen.pop();
```

Use `long long` for total, durations, and deadlines.

4.10 Python implementation notes

Use negative durations because `heapq` is a min-heap:

```
heapq.heappush(chosen, -duration)
if total > deadline:
    total += heapq.heappop(chosen)
```

The heap stores negative numbers, so popping and adding decreases the total.

4.11 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int solve_case(vector<pair<long long, long long>> tasks) {
    sort(tasks.begin(), tasks.end(), [](const auto &a, const auto &b) {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first;
    });
```

```

    long long total = 0;
    priority_queue<long long> chosen;
    for (auto [duration, deadline] : tasks) {
        total += duration;
        chosen.push(duration);
        if (total > deadline) {
            total -= chosen.top();
            chosen.pop();
        }
    }
    return (int)chosen.size();
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> tasks(n);
        for (auto &[duration, deadline] : tasks) cin >> duration >> deadline;
        cout << solve_case(tasks) << '\n';
    }

    return 0;
}

```

4.12 Full Python solution

```

import heapq
import sys

def solve_case(tasks: list[tuple[int, int]]) -> int:
    tasks.sort(key=lambda p: (p[1], p[0]))
    total = 0
    chosen: list[int] = []
    for duration, deadline in tasks:
        total += duration
        heapq.heappush(chosen, -duration)
        if total > deadline:
            total += heapq.heappop(chosen)
    return len(chosen)

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        tasks: list[tuple[int, int]] = []
        for _ in range(n):
            duration = data[idx]
            deadline = data[idx + 1]
            idx += 2
            tasks.append((duration, deadline))
        out.append(str(solve_case(tasks)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4.13 Common wrong answers

- Dropping the task with latest deadline. All processed deadlines are already at most the current deadline, so duration is what matters.
- Dropping the newest task. Sometimes the newest task is short and should stay.
- Sorting by duration only. Deadlines decide when constraints become active.
- Using a min-heap and accidentally removing the shortest task.

4.14 Tests that matter

Task too long for any deadline:
 (10,3), (10,4), (10,5) -> 0

New long task should be rejected:
 (1,3), (3,5), (2,6), (4,6) -> 3

Older long task should be replaced:
 (100,100), (1,2), (2,3), (2,4) -> 2

5 Stress testing greedy

Greedy solutions are excellent candidates for brute-force stress testing on small inputs.

For activity selection:

```
Try every subset of intervals.  
Check whether it is non-overlapping.  
Take the largest valid subset.
```

For rescue boats:

```
Use recursion or DP over remaining people.  
Try sending the heaviest alone or with each possible partner.
```

For deadline scheduling:

```
Try every subset.  
Sort the subset by deadline.  
Check whether every completion time meets its deadline.
```

The local random generators use exactly these ideas. That is why the reference checker is more valuable than fixed samples alone.

6 What to carry forward

Greedy problems become manageable when you separate three things:

1. the local choice;
2. the proof that the local choice is safe;
3. the data structure that implements repeated local choices efficiently.

For easy greedy problems, the proof may be one exchange. For harder Codeforces, AtCoder, and ICPC tasks, the proof may be an invariant that survives many local edits. The habit is the same: never trust the local rule until you can explain why an optimal solution can be made to follow it.